COMP 2011 Midterm Exam - **Fall** 2023 - HKUST

Date: October 28, 2023 (Saturday)
Time Allowed: 2 hours, 10:00-12:00noon
Instructions:

1. This is a closed-book, closed-notes examination.
2. There are **8** questions on **28** pages (including this cover page, honor code and 3 blank pages at the end).
3. Write your answers in the space provided in black/blue ink. NO pencil please, otherwise you are not allowed to appeal for any grading disagreements.
4. All programming codes in your answers must be written in the ANSI C++ version as taught in the class.
5. For programming questions, you are **NOT** allowed to define additional helper functions or structures, nor global variables unless otherwise stated. You **cannot** use any library functions not mentioned, nor the `auto` keyword in defining identifiers in the questions.

Student Name	
Student ID	
Email Address	
Lecture & Lab Section	

Problem	**Score**
1	/ 8
2	/ 8
3	/ 8
4	/ 18
5	/ 8
6	/ 10
7	/ 20
8	/ 20
Total	/ 100

The HKUST Academic Honor Code

Honesty and integrity are central to the academic work of HKUST. Students of the University must observe and uphold the highest standards of academic integrity and honesty in all the work they do throughout their program of study.

As members of the University community, students have the responsibility to help maintain the academic reputation of HKUST in its academic endeavors.

Sanctions will be imposed if students are found to have violated the regulations governing academic integrity and honesty.

Declaration of Academic Integrity

I confirm that I have answered the questions using only materials specifically approved for use in this examination, that all the answers are my own work, and that I have not received any assistance during the examination.

Signature: _____

Problem 1 [8 points] Parameter Passing

What is the output of the following code:

```
#include <iostream>
using namespace std;

int sum1(int &p, int &q ){
    p += 2;
    q = p;
    p += q;
    return p;
}

int & sum2(int &a, int b ){
    int temp = a;
    temp *= 2;
    a +=b;
    return a;
}

int main(void){
    int a = 1, b = 3, c=5;
    sum2(a,b);
    cout<<a<<" "<<b<<" "<<endl;
    sum1(a, sum2(a,b));
    cout<<a<<" "<<b<<" "<<endl;
    sum2(a, sum1(a,b));
    cout<<a<<" "<<b<<" "<<endl;
}
```

Answer:

Problem 2 [8 points] Mystery Recursion

For the following two parts (a) and (b), explain what the function `mystery` does using only one sentence. Be as precise as possible.

(a) *(4 points)*

```
int mystery(int x, int y) // assume x, y are non-negative
{
    if (x < y) return mystery(y, x);
    if (y != 0) return (x + mystery(x, y - 1));
    return 0;
}
```

Answer:

(b) *(4 points)*

```
void mystery(int a, int b)
{
    if (b<=0) return;
    if (a%b == 0) cout << b << " ";
    mystery(a,b-1);
    if (a%b == 0) cout << b << " ";

}
```

Answer:

Problem 3 [8 points] Scope

Consider the following program that compiles successfully:

```
#include <iostream>
using namespace std;

void foo(int& x) { x++; }

int main() {
  int a = 1;
  int& b = a;
  foo(b);

  if ((a--) == 2) {
    int a = 1;
    foo(a);
    cout << a << " " << b << endl;
  }

  cout << a << " " << b << endl;
  return 0;
}
```

What is the output of the program?

Answer:

Problem 4 [18 points] Print Shapes

Draw the shapes that are printed by the following programs.

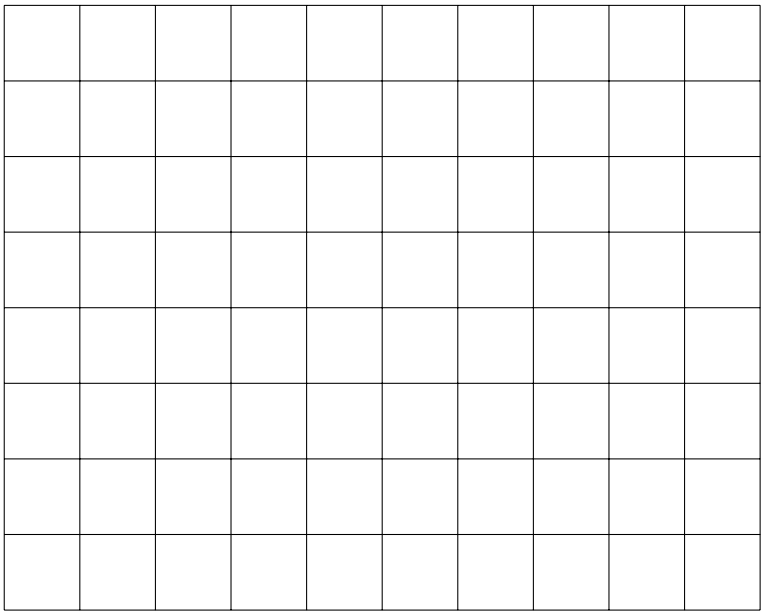
You can use the grid provided for better alignment. Note the grid space is more than enough. You only need to draw the visible characters. For an empty space, e.g. `cout << " ";`, you can just leave the corresponding grid cell empty. The printing should start from the leftmost part as if this is a terminal. You should not leave empty spaces on the left unless the code prints white spaces.

(a) *(5 points)*

```
#include <iostream>
using namespace std;

int main()
{
    int size = 4;
    for (int i = size; i > 0; i--) {
        for (int j = 1; j <= size; j++)
            if (j >= i)
                cout << "*";
            else
                cout << " ";
        cout << endl;
    }
    return 0;
}
```

Answer:



(b) *(6 points)*

```
#include <iostream>
using namespace std;

int main()
{
    int size = 4;
    for (int i = size; i >= 1; --i) {
        for (int k = 0; k < size - i; ++k)
            cout << " ";
        for (int j = i; j <= 2 * i - 1; ++j)
            cout << "*";
        for (int l = 0; l < i - 1; ++l)
            cout << "*";
        cout << endl;
    }
    return 0;
}
```

Answer:

(c) *(7 points)*

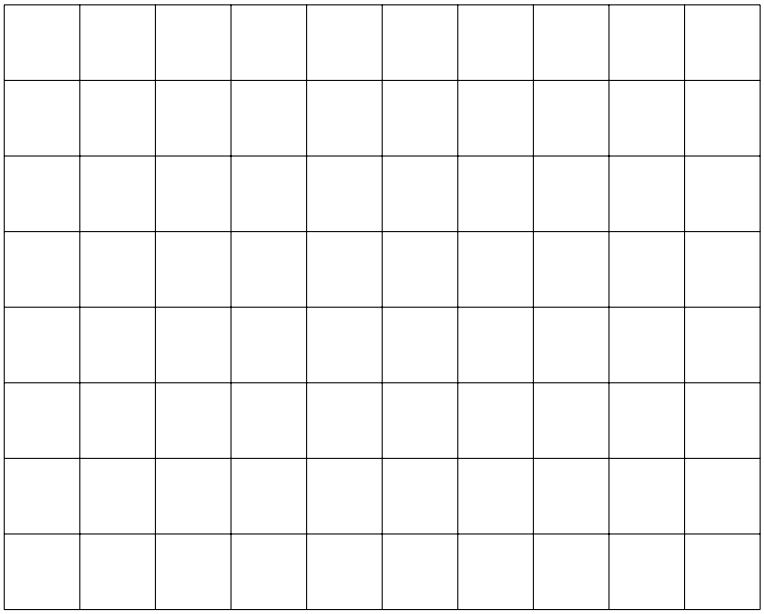
```
#include <iostream>
using namespace std;

int main()
{
    int size = 3, i = 1, j;
    while (i <= size) {
        j = size;

        while (j > i) {
            cout << " ";
            j--;
        }
        cout << "*";
        j = 1;
        while (j < (i - 1) * 2) {
            cout << " ";
            j++;
        }
        if (i == 1)
            cout << endl;
        else
            cout << "*" << endl;
        i++;
    }

    i = size - 1;
    while (i >= 1) {
        j = size;
        while (j > i) {
            cout << " ";
            j--;
        }
        cout << "*";
        j = 1;
        while (j < (i - 1) * 2) {
            cout << " ";
            j++;
        }
        if (i == 1)
            cout << endl;
        else
            cout << "*" << endl;
        i--;
    }
    return 0;
}
```

Answer:



Problem 5 [8 points] Print Matrix

What is the output of the following program?

```
#include <iostream>
using namespace std;

void matrixPrint(int a[][4], int m, int n)
{
    int i, k = 0, l = 0;

    while (k < m && l < n) {
        for (i = l; i < n; ++i)
            cout << a[k][i] << " ";
        k++;

        for (i = k; i < m; ++i)
            cout << a[i][n - 1] << " ";
        n--;

        if (k < m)
            for (i = n - 1; i >= l; --i)
                cout << a[m - 1][i] << " ";
            m--;

        if (l < n) {
            for (i = m - 1; i >= k; --i)
                cout << a[i][l] << " ";
            l++;
        }
    }
}

/* Driver Code */
int main()
{
    int a[4][4] = { { 1, 2, 3, 4 },
                    { 5, 6, 7, 8 },
                    { 9, 10, 11, 12 },
                    { 13, 14, 15, 16 } };

    // Function Call
    matrixPrint(a, 4, 4);
    return 0;
}
```

Answer: _____

Problem 6 [10 points] Function

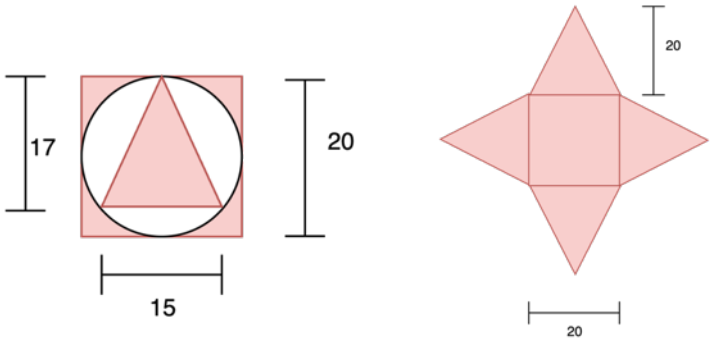
Recall that the formulae for computing the area of circle, triangle, and square are: Pi*Radius*Radius, $\frac{1}{2}$*base*height, and width*width, respectively.

Consider the following functions that return the area of each shape:

```
float circle(float radius);
float triangle(float base, float height);
float square(float width);
```

(a) *(4 points)*
Please provide the complete definition of the function triangle. This function should only take positive values as parameters. If any parameter is negative, it should return -1.

Answer:



(b) *(2 points)* Please provide **ONE** line of code in calling relevant functions to compute the shaded areas to complete the code below, using the labeled dimensions of the shapes.

`float shadedarealeft` = _____ (i)

Answer:

`float shadedarearight` = _____ (ii)

Answer:

(c) *(4 points)* It is possible to change the dimension first and then compute the area of shapes. Please study the function below:

```
int getArea() {
    float radius  = 3;
    /* Implement function doubleSize so that the next
       line doubles the radius three times */
    doubleSize(doubleSize(doubleSize(radius)));
    return circle(radius);
}
```

Provide the definition of the function `doubleSize` that doubles its input. This function should only take positive values as parameters. If any parameter is negative, it returns -1.

Answer:

Problem 7 [20 points] Struct

A store maintains inventory using a struct defined as:

```
struct Product
{
    int id;
    char name[20];
    int price;
    int quantity;
};
```

Complete the following functions for inventory control.

Important: Use pass by reference for structs wherever possible and constants for all pass by reference and array parameters that should not be modified by the function. No error checking of the input is required unless explicitly specified.

Note: You may use `strcpy(char dest[], char src[])`, which copies string `src` to `dest`. Finally, you may assume any needed standard header files and the namespace definition are included.

(a) *(7 points)* Write a function called `addProduct()` which takes the following parameters:

- `pId`, `pName`, `pPrice`, and `pQuantity` with the information of the new product.
- a `prods` array, which contains the product inventory and holds multiple objects of type product.
- `n`, which maintains the number of products currently in `prods`.
- `capacity`, which is the total (maximum) size of `prods`.

Your function should add the new product to the array, updating all needed variables to maintain the inventory. If a product with the same ID already exists, it should simply add `pQuantity` to the quantity (no further checking on name and price is needed). If the product cannot be added because the array is already filled to capacity, the function returns `false`, otherwise it returns `true`.

Here is an example code that calls `addProduct()`:

```
const int MAX_NUMBER = 3;   // maximum 3 products
int currentNumber = 0;      // the current number of products
Product prods[MAX_NUMBER];  // the array of products

// Example of adding the first product
if ( addProduct(1202, "Table", 34, 3, prods, currentNumber, MAX_NUMBER )) {
    cout << "Successful" << endl;
    currentNumber++;
} else {
    cout << "Failed" << endl;
}
```

```
__________ addProduct(_________ pId, _________ pName[], _________ pPrice,

_________ pQuantity, _________ prods[], _________ n, _________ capacity)
{









}
```

(b) *(3 points)* Write a function called `addItem()` which takes `pID`, `prods`, and `n`, and increments the quantity of the referred item in the array. If the item is not found, it just returns `false`, otherwise it returns `true`.

```
__________ addItem(__________ pId, __________ prods[], __________ n)
{









}
```

(c) *(3 points)* Write a function called `removeItem()` which takes `pID`, `prods`, and `n`, and decrements the quantity of the referred item in the array. If the item is not found or already has a quantity of zero, it just returns `false`, otherwise it returns `true`. Note: If after the update the quantity becomes zero, you should NOT remove the product; it stays in the array with a quantity of zero.

```
__________ removeItem(__________ pId, __________ prods[], __________ n)
{









}
```

(d) *(4 points)* Write a function called `removeProduct()` which takes `pID`, `prods`, and `n`, and removes the product from the array. The product should be removed regardless of the quantity of items available. When removing the product, you should move the last product to the position of the removed product in order not to leave a gap in the array. For example, if the array has 5 elements and you remove the 3rd element, you should move the 5th element to the position of the 3rd element and update the number of elements `n` to 4. If the item is not found, it just returns `false`, otherwise it returns `true`.

```
__________ removeProduct(__________ pId, __________ prods[], __________ &n)
{









}
```

(e) *(3 points)* Write a function called `printProduct()` to print the details of a given product. The function, which does not return any value, should take a product object `prod` and just output the information in the following format:

```
ID:1202
Name:Table
Price:34
Quantity:3
```

```
__________________ printProduct(_____________________ prod)
{




}
```

Problem 8 [20 points] C String

Please implement the indicated C string functions in the following program using 1-dimensional character array. Note that you are not allowed to use any other library such as `<cstring>` or `<string>`. Recall that a C string is always terminated by a null character.

```
#include <iostream>
using namespace std;

int strlen(const char string[]){
  int length = 0;
  for( ; string[length] != '\0'; length++)
    ;
  return length;
}

// delete the character at the index position
bool deleteChar(char string[], int position){
  // to be implemented
}

// insert newChar at index position
bool insertChar(char string[], char newChar, int position){
  // to be implemented
}

// find the first occurrence of word in string, and returns the index
int find(const char string[], const char word[]){
  // to be implemented
}

// replace word by repl in string
bool replace(char string[], const char word[], const char repl[]){
  // to be implemented
}

int main(){
  char str[20] = "COMP2011";  // C string
  char wrd1[] = "2011";  // C string
  char wrd2[] = " C++";  // C string

  // Using the functions above for C-string...

  return 0;
}
```

(a) *(4 points)* Implement the function `deleteChar`, which deletes from C string `string` the character at array index `position`. Note that the characters after the deletion point in the string have to be shifted one position to the left. Return `true` if the character is deleted; otherwise return `false` (for the case `string` is empty or `position` is not valid). For example, in the above main, a call of `deleteChar( str, 1 )` returns `true` and `str` becomes `CMP2011`, while a call of `deleteChar( str, 8 )` returns `false` and `str` is not changed.

Answer:

(b) *(4 points)* Implement the function `insertChar`, which inserts a new character `newChar` by putting it at array index `position` of the C string `string`. Please assume `string` has enough capacity to hold the extra character. Note that the original characters at or after the insertion point need to be shifted one position to the right. Return `true` if the insertion is successful, and `false` otherwise (`position` out of bound).

For example, in the above main, a call of `insertChar( str, ' ', 4 )` returns `true` and `str` becomes `COMP 2011`, while a call of `insertChar( str, 'C', 9 )` or `insertChar( str, 'C', -1 )` returns `false` and `str` is not changed.

Answer:

(c) *(6 points)* Implement the `find` function, which searches for the first occurrence of the C string `word` in the C string `string`. If there is a match, it returns the array index of `string` where the `word` begins. Otherwise, it returns -1.

For example, in the above main, a call of `find( str, wrd1 )` returns 4, while a call of `find( str, wrd2 )` returns `-1`.

Answers:

(d) *(6 points)* Implement the `replace` function, which replaces the first occurrence of the C-string word `word` in the C string `string` by the C-string word `repl`. It returns `true` if the replacement is executed, and `false` otherwise (i.e., `word` does not exist in `string`). Please assume `string` has enough capacity to hold `repl`. Please feel free to use the functions implemented in previous parts.

For example, in the above `main`, a call of `replace( str, wrd1, wrd2 )` returns `true` and `str` becomes `COMP C++`, while a call of `replace( str, wrd2, wrd1 )` returns `false` and `str` is not changed.

Answer:

---------- END OF PAPER ----------

/* Rough work — You may detach this page */

/* Rough work — You may detach this page */

/* Rough work — You may detach this page */

/* Rough work — You may detach this page */